

```

from __future__ import annotations

from datetime import datetime, timedelta
from pathlib import Path
import random

import pandas as pd

# Makes the generated dataset repeatable for demonstrations and evaluation.
SEED = 42
random.seed(SEED)

PROJECT_ROOT = Path(__file__).resolve().parents[1]
DATA_DIR = PROJECT_ROOT / "data"
START_DATE = datetime(2026, 6, 1, 8, 0, 0)

# City coordinates help the later feature-engineering stage calculate travel risk.
GEO_LOCATIONS = {
    "Bengaluru, IN": (12.9716, 77.5946),
    "Mumbai, IN": (19.0760, 72.8777),
    "Delhi, IN": (28.6139, 77.2090),
    "Hyderabad, IN": (17.3850, 78.4867),
    "London, UK": (51.5072, -0.1276),
    "New York, US": (40.7128, -74.0060),
    "Singapore, SG": (1.3521, 103.8198),
    "Tokyo, JP": (35.6762, 139.6500),
}

USER_RESOURCES = [
    "employee_portal",
    "email",
    "crm",
    "project_share",
    "analytics_dashboard",
]

SERVICE_RESOURCES = [
    "api_gateway",
    "database_read",
    "telemetry_queue",
    "backup_service",
]

DEVICE_RESOURCES = [
    "telemetry_endpoint",
    "firmware_status",
    "device_health",
    "sensor_stream",
]

PRIVILEGED_RESOURCES = [
    "admin_console",
    "domain_controller",
    "backup_vault",
    "hr_records",
    "finance_share",
]

SENSITIVE_RESOURCES = [
    "customer_export",
    "finance_export",
    "source_code_archive",
]

def private_ip() -> str:
    """Create an internal-looking IP address."""
    return f"10.{random.randint(1, 250)}.{random.randint(0, 250)}.{random.randint(1, 254)}"

def attacker_ip() -> str:
    """Use documentation-only public IP ranges for simulated attacks."""
    prefix = random.choice(["198.51.100", "203.0.113"])
    return f"{prefix}.{random.randint(1, 254)}"

def build_profiles() -> list[dict]:
    """Create baseline behavioural profiles for users, services, and devices."""
    profiles = []

    entity_types = (
        ["user"] * 45
        + ["service_account"] * 10
        + ["edge_device"] * 10
    )

    for index, entity_type in enumerate(entity_types, start=1):
        if entity_type == "user":
            entity_id = f"user_{index:03d}"
            resources = random.sample(USER_RESOURCES, k=3)
            auth_methods = ["password", "token"]
            fingerprint = random.choice(
                [
                    "Windows-11|Chrome|managed",
                    "Windows-11|Edge|managed",
                    "macOS-15|Chrome|managed",
                ]
            )
            commands = ["login", "open_dashboard", "read_resource", "logout"]
        elif entity_type == "service_account":
            entity_id = f"svc_{index:03d}"
            resources = random.sample(SERVICE_RESOURCES, k=3)
            auth_methods = ["token", "certificate"]
            fingerprint = "Linux-Server|service-agent"
            commands = ["token_auth", "api_read", "api_write", "logout"]
        else:
            entity_id = f"device_{index:03d}"
            resources = random.sample(DEVICE_RESOURCES, k=3)
            auth_methods = ["certificate"]
            fingerprint = random.choice(
                [
                    "IoT-Linux-5.10|MQTT|device",
                    "EdgeOS-3.2|HTTPS|device",
                ]
            )
            commands = ["device_connect", "telemetry_send", "heartbeat"]

        profiles.append(
            {
                "entity_id": entity_id,
                "entity_type": entity_type,
                "home_location": random.choice(list(GEO_LOCATIONS)),
                "usual_hour": random.randint(8, 18),
                "base_ip": private_ip(),
            }
        )

```

```

        "normal_resources": resources,
        "auth_methods": auth_methods,
        "device_fingerprint": fingerprint,
        "normal_commands": commands,
    }
)

return profiles

def make_event(
    profile: dict,
    timestamp: datetime,
    *,
    source_ip: str | None = None,
    geo_location: str | None = None,
    resource: str | None = None,
    auth_method: str | None = None,
    auth_result: str = "success",
    session_duration: int | None = None,
    command_sequence: list[str] | None = None,
    device_fingerprint: str | None = None,
    bytes_transferred: int | None = None,
    protocol: str = "HTTPS",
    label: str = "normal",
    anomaly_type: str = "none",
    attack_notes: str = "Expected behavioural baseline",
    is_anomaly: int | None = None,
) -> dict:
    """Create one event that matches the required access-log schema."""
    location = geo_location or profile["home_location"]
    latitude, longitude = GEO_LOCATIONS[location]

    if is_anomaly is None:
        is_anomaly = int(label != "normal")

    return {
        "entity_id": profile["entity_id"],
        "entity_type": profile["entity_type"],
        "timestamp": timestamp,
        "source_ip": source_ip or profile["base_ip"],
        "geo_location": location,
        "geo_lat": latitude,
        "geo_lon": longitude,
        "resource_accessed": resource or random.choice(profile["normal_resources"]),
        "auth_method": auth_method or random.choice(profile["auth_methods"]),
        "auth_result": auth_result,
        "session_duration": session_duration
        if session_duration is not None
        else random.randint(5, 180),
        "command_sequence": " " > ".join(
            command_sequence or profile["normal_commands"]
        ),
        "device_fingerprint": device_fingerprint
        or profile["device_fingerprint"],
        "bytes_transferred": bytes_transferred
        if bytes_transferred is not None
        else random.randint(5_000, 500_000),
        "protocol": protocol,
        "label": label,
        "is_anomaly": is_anomaly,
        "anomaly_type": anomaly_type,
        "attack_notes": attack_notes,
    }

def normal_timestamp(profile: dict) -> datetime:
    """Generate a normal time near the entity's usual access hour."""
    day_offset = random.randint(0, 29)
    hour = max(0, min(23, round(random.gauss(profile["usual_hour"], 1.4))))
    return START_DATE + timedelta(
        days=day_offset,
        hours=hour - START_DATE.hour,
        minutes=random.randint(0, 59),
        seconds=random.randint(0, 59),
    )

def generate_normal_events(profiles: list[dict]) -> list[dict]:
    events = []

    for profile in profiles:
        for _ in range(60):
            result = "success" if random.random() < 0.97 else "failure"
            events.append(
                make_event(
                    profile,
                    normal_timestamp(profile),
                    auth_result=result,
                    session_duration=0 if result == "failure" else None,
                    attack_notes="Expected behavioural baseline",
                )
            )

    return events

def inject_brute_force(events: list[dict], users: list[dict]) -> None:
    """Repeated failed authentication attempts against one account."""
    for campaign in range(4):
        target = random.choice(users)
        attack_time = START_DATE + timedelta(
            days=random.randint(20, 28),
            hours=random.randint(0, 4),
            minutes=random.randint(0, 30),
        )
        source = attacker_ip()

        for attempt in range(12):
            events.append(
                make_event(
                    target,
                    attack_time + timedelta(seconds=attempt * 12),
                    source_ip=source,
                    resource="vpn_gateway",
                    auth_method="password",
                    auth_result="failure",
                    session_duration=0,
                    command_sequence=["login_failed"],
                    bytes_transferred=0,
                    label="anomaly",
                    anomaly_type="brute_force",
                    attack_notes="12 failed password attempts from one IP in a short window",
                )
            )

```

```

    )
)

def inject_impossible_travel(events: list[dict], users: list[dict]) -> None:
    """A valid login from a distant location minutes after a normal login."""
    for _ in range(12):
        target = random.choice(users)
        first_time = START_DATE + timedelta(
            days=random.randint(21, 28),
            hours=random.randint(8, 18),
            minutes=random.randint(0, 30),
        )
        distant_location = random.choice(
            [
                city
                for city in GEO_LOCATIONS
                if city != target["home_location"]
            ]
        )

        events.append(
            make_event(
                target,
                first_time,
                attack_notes="Normal login preceding the impossible-travel event",
            )
        )
        events.append(
            make_event(
                target,
                first_time + timedelta(minutes=25),
                source_ip=attacker_ip(),
                geo_location=distant_location,
                resource="employee_portal",
                label="anomaly",
                anomaly_type="impossible_travel",
                attack_notes="Distant locations accessed only 25 minutes apart",
            )
        )

def inject_credential_stuffing(events: list[dict], users: list[dict]) -> None:
    """One source IP tries many accounts with a high failure rate."""
    for campaign in range(3):
        source = attacker_ip()
        attack_time = START_DATE + timedelta(
            days=random.randint(18, 29),
            hours=random.randint(0, 5),
        )

        for offset, target in enumerate(random.sample(users, k=20)):
            events.append(
                make_event(
                    target,
                    attack_time + timedelta(seconds=offset * 20),
                    source_ip=source,
                    resource="vpn_gateway",
                    auth_method="password",
                    auth_result="failure",
                    session_duration=0,
                    command_sequence=["login_failed"],
                    bytes_transferred=0,
                    label="anomaly",
                    anomaly_type="credential_stuffing",
                    attack_notes="One source IP attempted many entity accounts",
                )
            )

def inject_lateral_movement(events: list[dict], users: list[dict]) -> None:
    """A compromised user reaches privileged resources never normally used."""
    for _ in range(6):
        target = random.choice(users)
        attack_time = START_DATE + timedelta(
            days=random.randint(20, 29),
            hours=random.randint(19, 23),
        )

        for step, resource in enumerate(random.sample(PRIVILEGED_RESOURCES, k=5)):
            events.append(
                make_event(
                    target,
                    attack_time + timedelta(minutes=step * 4),
                    source_ip=attacker_ip(),
                    resource=resource,
                    command_sequence=[
                        "login",
                        "enumerate_hosts",
                        "access_privileged_resource",
                        "logout",
                    ],
                    bytes_transferred=random.randint(500_000, 2_000_000),
                    label="anomaly",
                    anomaly_type="lateral_movement",
                    attack_notes="Unusual sequence and breadth of privileged resources",
                )
            )

def inject_device_spoofing(events: list[dict], users: list[dict]) -> None:
    """A known account appears with an unseen device fingerprint."""
    for target in random.sample(users, k=15):
        attack_time = START_DATE + timedelta(
            days=random.randint(15, 29),
            hours=random.randint(0, 6),
        )

        events.append(
            make_event(
                target,
                attack_time,
                source_ip=attacker_ip(),
                device_fingerprint="Unknown-OS|Unmanaged|fingerprint-mismatch",
                label="anomaly",
                anomaly_type="device_spoofing",
                attack_notes="Entity ID observed with a device fingerprint not in its history",
            )
        )

def inject_low_and_slow_exfiltration(events: list[dict], users: list[dict]) -> None:
    """Small but suspicious off-hours exports spread across days."""
    for _ in range(8):

```

```

target = random.choice(users)
starting_day = random.randint(14, 24)

for day_offset in range(5):
    events.append(
        make_event(
            target,
            START_DATE + timedelta(
                days=starting_day + day_offset,
                hours=random.randint(-7, -4),
                minutes=random.randint(0, 59),
            ),
            resource=random.choice(SENSITIVE_RESOURCES),
            command_sequence=[
                "login",
                "query_sensitive_data",
                "compress_archive",
                "upload_external",
            ],
            bytes_transferred=random.randint(2_000_000, 5_000_000),
            label="anomaly",
            anomaly_type="low_and_slow_exfiltration",
            attack_notes="Repeated off-hours sensitive-data transfer building over days",
        )
    )

def inject_insider_drift(events: list[dict], users: list[dict]) -> None:
    """Benign edge cases used later to tune false-positive behaviour."""
    for target in random.sample(users, k=8):
        starting_day = random.randint(10, 20)

        for day_offset, resource in enumerate(
            random.sample(PRIVILEGED_RESOURCES, k=5)
        ):
            events.append(
                make_event(
                    target,
                    START_DATE + timedelta(
                        days=starting_day + day_offset * 2,
                        hours=target["usual_hour"] - START_DATE.hour,
                    ),
                    resource=resource,
                    command_sequence=[
                        "login",
                        "approved_new_access",
                        "read_resource",
                        "logout",
                    ],
                    bytes_transferred=random.randint(100_000, 400_000),
                    label="normal",
                    anomaly_type="insider_drift",
                    attack_notes="Benign edge case: gradual approved resource expansion",
                    is_anomaly=0,
                )
            )

def inject_cold_start_entities(
    events: list[dict], users: list[dict]
) -> list[dict]:
    """Introduce new legitimate users after the baseline period.

    These records validate that population-profile scoring handles a brand-new
    identity without treating its first normal activities as an intrusion.
    """
    new_profiles = []
    for index in range(1, 6):
        template = random.choice(users)
        profile = {
            **template,
            "entity_id": f"new_user_{index:03d}",
            "base_ip": private_ip(),
        }
        new_profiles.append(profile)
        for day_offset in range(5):
            events.append(
                make_event(
                    profile,
                    START_DATE
                    + timedelta(
                        days=23 + day_offset,
                        hours=profile["usual_hour"] - START_DATE.hour,
                        minutes=random.randint(0, 59),
                    ),
                    attack_notes="Legitimate new user for cold-start evaluation",
                )
            )
    return new_profiles

def main() -> None:
    DATA_DIR.mkdir(exist_ok=True)

    profiles = build_profiles()
    users = [p for p in profiles if p["entity_type"] == "user"]

    events = generate_normal_events(profiles)

    # Inject the seven behaviours specified in the assessment brief.
    inject_brute_force(events, users)
    inject_impossible_travel(events, users)
    inject_credential_stuffing(events, users)
    inject_lateral_movement(events, users)
    inject_device_spoofing(events, users)
    inject_low_and_slow_exfiltration(events, users)
    inject_insider_drift(events, users)
    profiles.extend(inject_cold_start_entities(events, users))

    df = pd.DataFrame(events).sort_values("timestamp").reset_index(drop=True)
    df.insert(0, "event_id", [f"evT_{index:06d}" for index in range(1, len(df) + 1)])

    logs_path = DATA_DIR / "synthetic_access_logs.csv"
    df.to_csv(logs_path, index=False)

    profile_rows = []
    for profile in profiles:
        profile_rows.append(
            {
                key: " | ".join(value) if isinstance(value, list) else value
                for key, value in profile.items()
            }
        )

```

```

profiles_path = DATA_DIR / "entity_profiles.csv"
pd.DataFrame(profile_rows).to_csv(profiles_path, index=False)

print(f"Created {len(df):,} access events.")
print(f"Saved logs to: {logs_path}")
print(f"Saved profiles to: {profiles_path}")
print("\nDataset distribution:")
print(
    df.groupby(["label", "anomaly_type"])
      .size()
      .reset_index(name="events")
      .to_string(index=False)
)

if __name__ == "__main__":
    main()

```